



University Institute of Engineering

Department of Computer Science & Engineering

COMPUTER ORGANIZATION & ARCHITECTURE

(23CST-204/23ITT-204)

ER. SHIKHA ATWAL

E11186

ASSISTANT PROFESSOR

BE-CSE

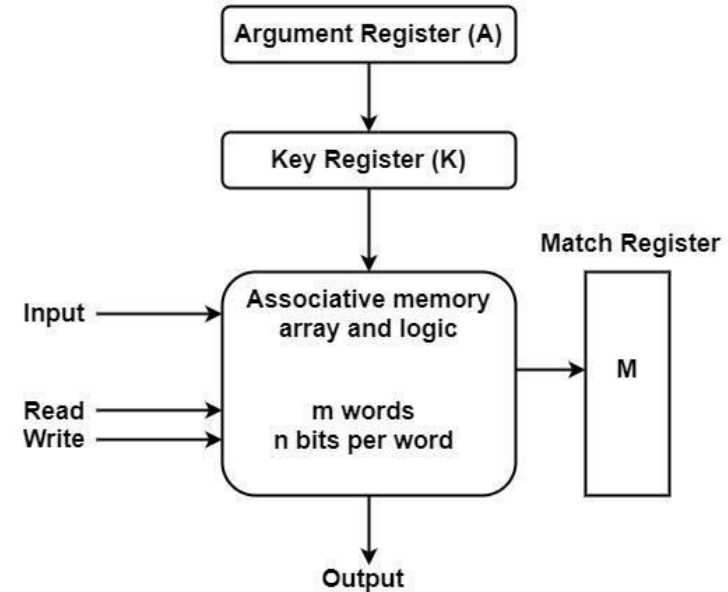
ASSOCIATIVE MEMORY

An associative memory can be treated as a memory unit whose saved information can be recognized for approach by the content of the information itself instead of by an address or memory location. Associative memory is also known as **Content Addressable Memory (CAM)**.

The block diagram of associative memory is shown in the figure above. It consists of a memory array and logic for m words with n bits per word.

The argument register A and key register K each have n bits, one for each bit of a word. The match register M has m bits, one for each memory word. Each word in memory is compared in parallel with the content of the argument register. The words that match the bits of the argument register set a corresponding bit in the match register.

Block Diagram of Associative Memory



After the matching process, those bits in the match register that have been set indicate the fact that their corresponding words have been matched.

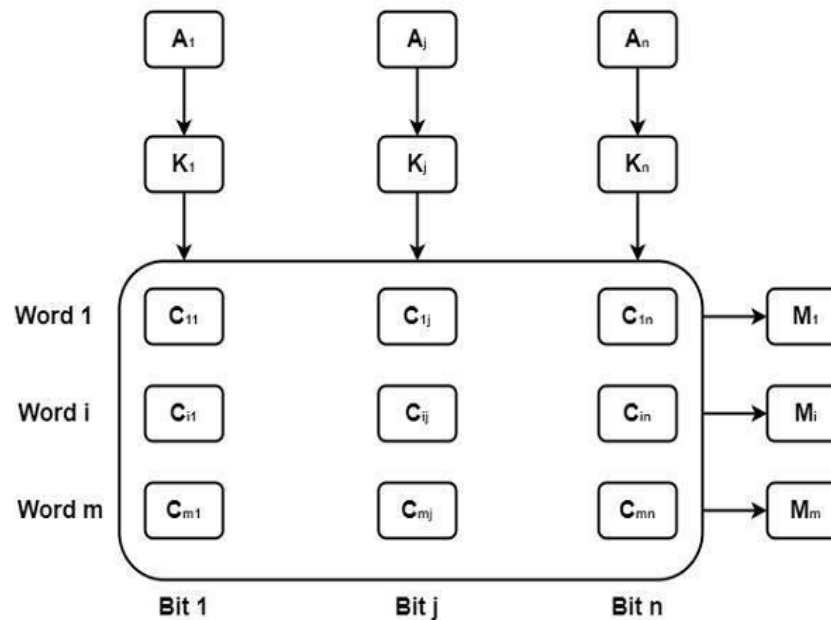
Reading is accomplished by a sequential access to memory for those words whose corresponding bits in the match register have been set.

The key register supports a mask for selecting a specific field or key in the argument word. The whole argument is distinguished with each memory word if the key register includes all 1's.

Hence, there are only those bits in the argument that have 1's in their equivalent position of the key register are compared. Therefore, the key gives a mask or recognizing a piece of data that determines how the reference to memory is created.

The following figure can define the relation between the memory array and the external registers in associative memory.

Associative Memory of m word, n cells per word



The cells in the array are considered by the letter C with two subscripts. The first subscript provides the word number and the second determines the bit position in the word. Therefore, cell c_{ij} is the cell for bit j in word i .

A bit A_j in the argument register is compared with all the bits in column j of the array supported that $K_j = 1$. This is completed for all columns $j = 1, 2, \dots, n$.

If a match appears between all the unmasked bits of the argument and the bits in word I , the corresponding bit M_i in the match register is set to 1. If one or more unmasked bits of the argument and the word do not match, M_i is cleared to 0.

Applications of Associative memory:

1. It can be only used in memory allocation format.
2. It is widely used in the database management systems, etc.
3. Networking: Associative memory is used in network routing tables to quickly find the path to a destination network based on its address.
4. Image processing: Associative memory is used in image processing applications to search for specific features or patterns within an image.
5. Artificial intelligence: Associative memory is used in artificial intelligence applications such as expert systems and pattern recognition.
6. Database management: Associative memory can be used in database management systems to quickly retrieve data based on its content.

Advantages of Associative memory:

1. It is used where search time needs to be less or short.
2. It is suitable for parallel searches.
2. It is often used to speedup databases.
3. It is used in page tables used by the virtual memory and used in neural networks.

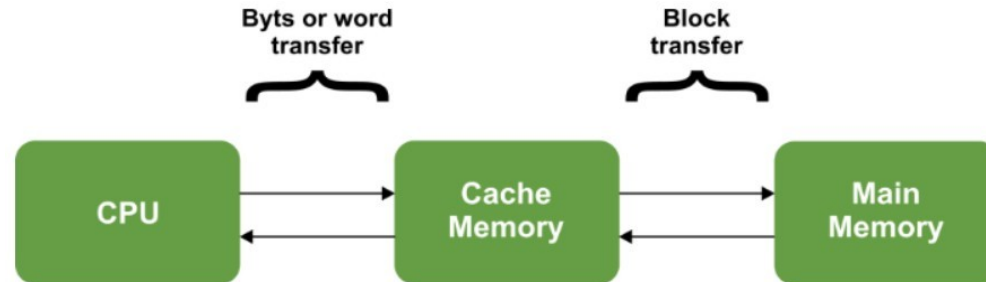
Disadvantages of Associative memory:

4. It is more expensive than RAM
5. Each cell must have storage capability and logical circuits for matching its content with external argument.

CACHE MEMORY

The data or contents of the main memory that are used frequently by CPU are stored in the cache memory so that the processor can easily access that data in a shorter time. Whenever the CPU needs to access memory, it first checks the cache memory. If the data is not found in cache memory, then the CPU moves into the main memory.

Cache memory is placed between the CPU and the main memory. The block diagram for a cache memory can be represented as:



The cache is the fastest component in the memory hierarchy and approaches the speed of CPU components.

Need of cache memory

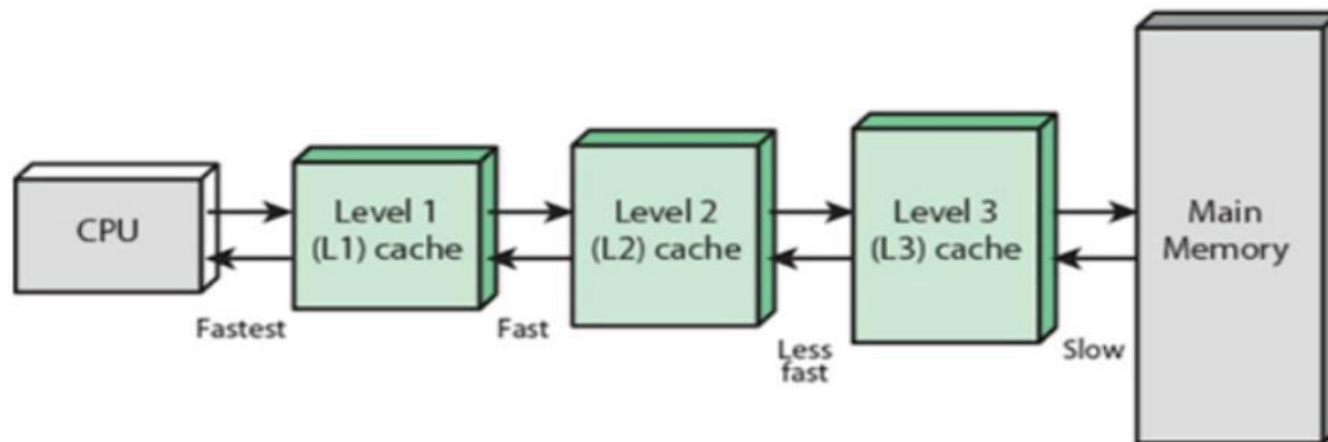
Data in primary memory can be accessed faster than secondary memory but still, access times of primary memory are generally in few microseconds, whereas CPU is capable of performing operations in nanoseconds. Due to the time lag between accessing data and acting of data performance of the system decreases as the CPU is not utilized properly, it may remain idle for some time. In order to minimize this time gap new segment of memory is Introduced known as Cache Memory.

Types of Cache Memory

L1 or Level 1 Cache: It is the first level of cache memory that is present inside the processor. It is present in a small amount inside every core of the processor separately. The size of this memory ranges from 2KB to 64 KB.

L2 or Level 2 Cache: It is the second level of cache memory that may present inside or outside the CPU. If not present inside the core, It can be shared between two cores depending upon the architecture and is connected to a processor with the high-speed bus. The size of memory ranges from 256 KB to 512 KB.

L3 or Level 3 Cache: It is the third level of cache memory that is present outside the CPU and is shared by all the cores of the CPU. Some high processors may have this cache. This cache is used to increase the performance of the L2 and L1 cache. The size of this memory ranges from 1 MB to 8MB.



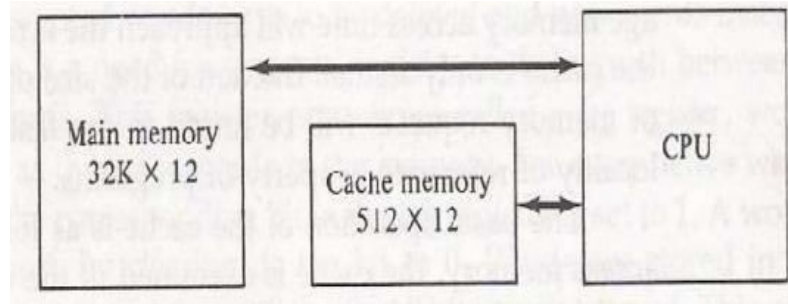
Locality of Reference

- Effectiveness of cache mechanism is based on a property of computer programs called “locality of reference”.
- The references to memory at any given time interval tend to be confined within a localized area.
- Analysis of programs shows that most of their execution time is spent on routines in which instructions are executed repeatedly. These instructions may be – loops, nested loops, or few procedures that call each other.
- Many instructions in localized areas of program are executed repeatedly during some time period and remainder of the program is accessed infrequently.
- This property is called “Locality of Reference”.

Locality of reference is manifested in two ways:

1. **Temporal** means that a recently executed instruction is likely to be executed again very soon. The information which will be used in near future is likely to be in use already (e.g. reuse of information in loops)
2. **Spatial** means that instructions in close proximity to a recently executed instruction are also likely to be executed soon. If a word is accessed, adjacent (near) words are likely to be accessed soon (e.g. related data items (arrays) are usually stored together; instructions are executed sequentially)

Principles of cache



The main memory can store 32k words of 12 bits each. The cache is capable of storing 512 of these words at any given time. For every word stored, there is a duplicate copy in main memory. The CPU communicates with both memories. It first sends a 15-bit address to cache. If there is a hit, the CPU accepts the 12-bit data from cache. If there is a miss, the CPU reads the word from main memory and the word is then transferred to cache.

- When a read request is received from CPU, contents of a block of memory words containing the location specified are transferred in to cache.
- When the program references any of the locations in this block, the contents are read from the cache. Number of blocks in cache is smaller than number of blocks in main memory.
- Correspondence between main memory blocks and those in the cache is specified by a mapping function.
- Assume cache is full and memory word not in cache is referenced.
- Control hardware decides which block from cache is to be removed to create space for new block containing referenced word from memory.
- Collection of rules for making this decision is called “**Replacement algorithm**”

Cache performance

- On searching in the cache if data is found, a cache hit has occurred.
- On searching in the cache if data is not found, a cache miss has occurred.

Performance of cache is measured by the number of cache hits to the number of searches. This parameter of measuring performance is known as the Hit Ratio.

Hit ratio = (Number of cache hits)/(Number of searches)

Cache Mapping

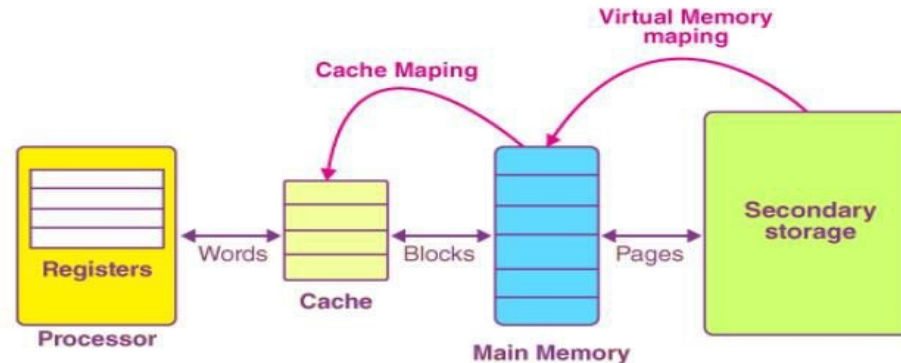
As we know that the cache memory bridges the mismatch of speed between the main memory and the processor. Whenever a cache hit occurs,

- The word that is required is present in the memory of the cache. Then the required word would be delivered from the cache memory to the CPU.
- And, whenever a cache miss occurs, the word that is required isn't present in the memory of the cache. The page consists of the required word that we need to map from the main memory.
- We can perform such a type of mapping using various different techniques of cache mapping.

Process of Cache Mapping

The process of cache mapping helps us define how a certain block that is present in the main memory gets mapped to the memory of a cache in the case of any cache miss.

In simpler words, cache mapping refers to a technique using which we bring the main memory into the cache memory. Here is a diagram that illustrates the actual process of mapping:



Important Note:

- The main memory gets divided into multiple partitions of equal size, known as the frames or blocks.
- The cache memory is actually divided into various partitions of the same sizes as that of the blocks, known as lines.
- The main memory block is copied simply to the cache during the process of cache mapping, and this block isn't brought at all from the main memory.

Cache Mapping Functions

Correspondence between main memory blocks and those in the cache is specified by a memory mapping function.

There are three techniques in memory mapping

- 1.Direct Mapping
- 2.Fully Associative Mapping
- 3.Set Associative Mapping

Direct Mapping

In direct mapping, the cache consists of normal high-speed random-access memory. A certain block of the main memory would be able to map a cache only up to a certain line of the cache. The total line numbers of cache to which any distinct block can map are given by the following:

Cache line number = (Address of the Main Memory Block) Modulo (Total number of lines in Cache)

For example,

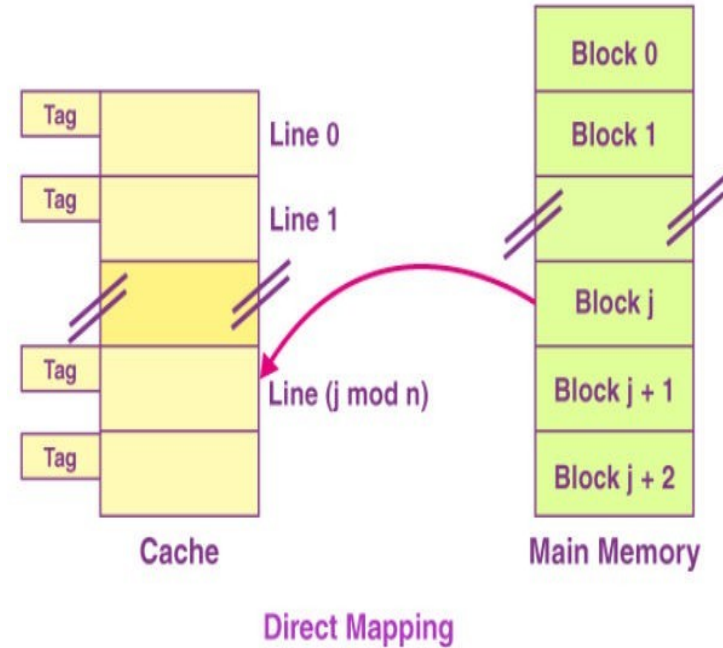
Let us consider that particular cache memory is divided into a total of 'n' number of lines.

Then, the block 'j' of the main memory would be able to map to line number only of the cache ($j \bmod n$).

The Need for Replacement Algorithm

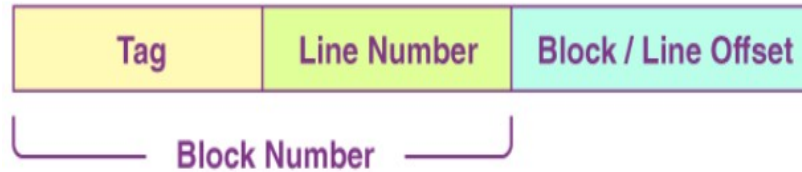
In the case of direct mapping,

- There is no requirement for a replacement algorithm.
- It is because the block of the main memory would be able to map to a certain line of the cache only.
- Thus, the incoming (new) block always happens to replace the block that already exists, if any, in this certain line.



Division of Physical Address

In the case of direct mapping, the division of the physical address occurs as follows:

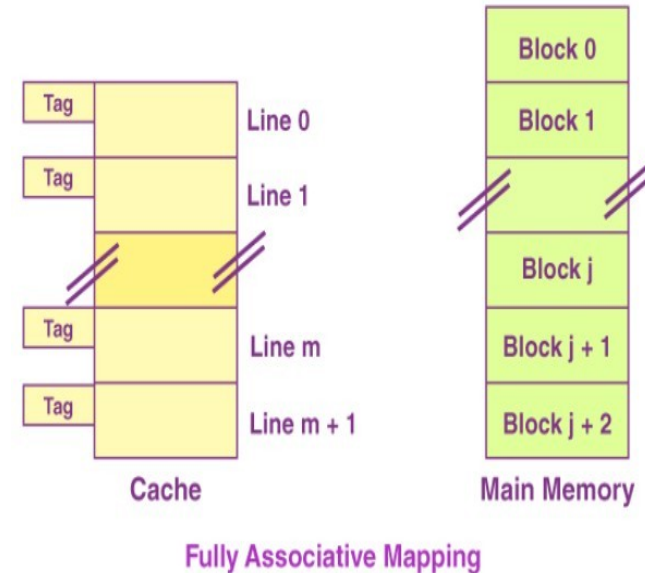


Fully Associative Mapping

In the case of fully associative mapping,

- The main memory block is capable of mapping to any given line of the cache that's available freely at that particular moment.
- It helps us make a fully associative mapping comparatively more flexible than direct mapping.

Let us consider the scenario given as follows:



Here, we can see that,

- Every single line of cache is available freely.
- Thus, any main memory block can map to a line of the cache.
- In case all the cache lines are occupied, one of the blocks that exists already needs to be replaced.

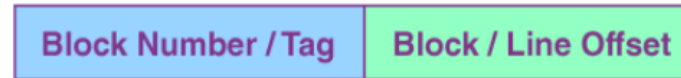
The Need for Replacement Algorithm

In the case of fully associative mapping,

- The replacement algorithm is always required.
- The replacement algorithm suggests a block that is to be replaced whenever all the cache lines happen to be occupied.
- So replacement algorithms such as LRU Algorithm, FCFS Algorithm, etc. are employed.

Division of Physical Address

In the case of fully associative mapping, the division of the physical address occurs as follows:



K-way Set Associative Mapping

In the case of k-way set associative mapping,

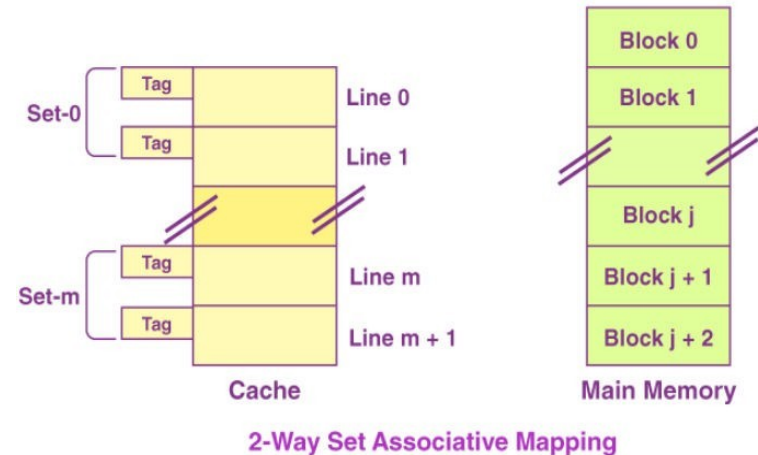
- The grouping of the cache lines occurs into various sets where all the sets consist of k number of lines.
- Any given main memory block can map only to a particular cache set.
- However, within that very set, the block of memory can map any cache line that is freely available.
- The cache set to which a certain main memory block can map is basically given as follows:

Cache set number = (Block Address of the Main Memory) Modulo (Total Number of sets present in the Cache)

Let us consider the example given as follows of a two-way set-associative mapping:

In this case,

- $k = 2$ would suggest that every set consists of two cache lines.
- Since the cache consists of 6 lines, the total number of sets that are present in the cache = $6/2 = 3$ sets.
- The block 'j' of the main memory is capable of mapping to the set number only ($j \bmod 3$) of the cache.
- Here, within this very set, the block 'j' is capable of mapping to any cache line that is freely available at that moment.
- In case all the available cache lines happen to be occupied, then one of the blocks that already exist needs to be replaced.



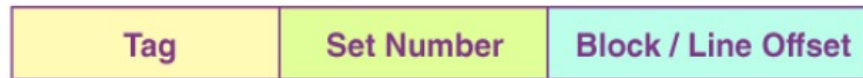
The Need for Replacement Algorithm

In the case of k-way set associative mapping,

- The k-way set associative mapping refers to a combination of the direct mapping as well as the fully associative mapping.
- It makes use of the fully associative mapping that exists within each set.
- Therefore, the k-way set associative mapping needs a certain type of replacement algorithm.

Division of Physical Address

In the case of fully k-way set mapping, the division of the physical address occurs as follows:



Special Cases

- In case $k = 1$, the k -way set associative mapping would become direct mapping. Thus, Direct Mapping = one-way set associative mapping
- In the case of $k =$ The total number of lines present in the cache, then the k -way set associative mapping would become fully associative mapping.

CACHE REPLACEMENT POLICIES

Replacement algorithms are used when there is no available space in a cache in which to place a data.

A page replacement algorithm is needed to decide which page needs to be replaced when the new page comes in. Whenever a new page is referred to and is not present in memory, the page fault occurs and the Operating System replaces one of the existing pages with a newly needed page.

Four of the most common cache replacement algorithms are described below:

1.FIFO (First In First Out) Policy

- The block which has entered first in the main be replaced first.
- This can lead to a problem known as "**Belady's Anomaly**", it starts that if we increase the no. of lines in cache memory the cache miss will increase.
- **Belady's Anomaly:** For some cache replacement algorithm, the page fault or miss rate increase as the number of allocated frame increase.
- Example: Let we have a sequence 7, 0 ,1, 2, 0, 3, 0, 4, 2, 3, and cache memory has 4 lines.

7	0	1	2	0	3	0	4	2	3
7	7	7	7		3		3		
	0	0	0		0		4		
		1	1		1		1		
			2		2		2		
m	m	m	m	H	m	H	m	H	H

There are a total of 6 misses in the **FIFO** replacement policy.

2. LRU (Least Recently Used)

- The page which was not used for the largest period of time in the past will get reported first.
- We can think of this strategy as the optimal cache- replacement algorithm looking backward in time, rather than forward.
- LRU is much better than FIFO replacement.
- LRU is also called a stack algorithm and can never exhibit belady's anomaly.
- The problem which is most important is how to implement LRU replacement. An LRU page replacement algorithm may require a sustainable hardware resource.
- **Example:** Let we have a sequence 7, 0, 1, 2, 0, 3, 0, 4, 2, 3 and cache memory has 3 lines.

7	0	1	2	0	3	0	4	2	3
7	7	7	2		2		4		4
	0	0	0		0		0		3
		1	1		3		2		2
M	M	M	M	H	M	H	M	H	M

There are a total of 6 misses in the **LRU replacement policy**.

3. LFU (Least Frequently Used):

This cache algorithm uses a counter to keep track of how often an entry is accessed. With the LFU cache algorithm, the entry with the lowest count is removed first. This method isn't used that often, as it does not account for an item that had an initially high access rate and then was not accessed for a long time.

4. Random Replacement (RR):

This algorithm randomly selects an object when it reaches maximum capacity. It has the benefit of not keeping any reference or history of objects and being very simple to implement at the same time.

This algorithm has been used in ARM processors and the famous Intel i860.

Cache Design Issues

1.Cache Addresses:

- Logical Cache/Virtual Cache stores data using virtual addresses. It accesses cache directly without going through MMU
- Physical Cache stores data using main memory physical addresses.

One obvious advantage of the logical cache is that cache access speed is faster than for a physical cache, because the cache can respond before the MMU performs an address translation.

The disadvantage has to do with the fact that most virtual memory systems supply each application with the same virtual memory address space. That is, each application sees a virtual memory that starts at address 0. Thus, the same virtual address in two different applications refers to two different physical addresses. The cache memory must therefore be completely flushed with each application switch, or extra bits must be added to each line of the cache to identify which virtual address space this address refers to.

2. Cache Size

The larger the cache, the larger the number of gates involved in addressing the cache. The available chip and board area also limit cache size.

The more cache a system has, the more likely it is to register a hit on memory access because fewer memory locations are forced to share the same cache line.

Although an increase in cache size will increase the hit ratio, a continuous increase in cache size will not yield an equivalent increase of the hit ratio.

***Note:** An Increase in cache size from 256K to 512K (increase by 100%) will yield a 10% improvement of the hit ratio, but an additional increase from 512K to 1024K would yield a less than 5% increase of the hit ratio (law of diminishing marginal returns).*

3. Replacement Algorithm

Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced.

For direct mapping, there is only one possible line for any particular block, and no choice is possible.

Direct mapping — No choice, Each block only maps to one line. Replace that line.

For the associative and set-associative techniques, a replacement algorithm is needed. To achieve high speed, such an algorithm must be implemented in hardware.

Least Recently Used (LRU) — Most Effective

3. Replacement Algorithm

For two- way set associative, this is easily implemented. Each line includes a USE bit. When a line is referenced, its USE bit is set to 1 and the USE bit of the other line in that set is set to 0. When a block is to be read into the set, the line whose USE bit is 0 is used.

Because we are assuming that more recently used memory locations are more likely to be referenced, LRU should give the best hit ratio. LRU is also relatively easy to implement for a fully associative cache. The cache mechanism maintains a separate list of indexes to all the lines in the cache. When a line is referenced, it moves to the front of the list. For replacement, the line at the back of the list is used. Because of its simplicity of implementation, LRU is the most popular replacement algorithm.

4. Write Policy

When you are saving changes to main memory. There are two techniques involved:

Write Through:

- Every time an operation occurs, you store to main memory as well as cache simultaneously. Although that may take longer, it ensures that main memory is always up to date and this would decrease the risk of data loss if the system would shut off due to power loss. This is used for highly sensitive information.
- One of the central caching policies is known as write-through. This means that data is stored and written into the cache and to the primary storage device at the same time.

- One advantage of this policy is that it ensures information will be stored safely without risk of data loss. If the computer crashes or the power goes out, data can still be recovered without issue.
- To keep data safe, this policy has to perform every write operation twice. The program or application that is being used must wait until the data has been written to both the cache and storage device before it can proceed.
- This comes at the cost of system performance but is highly recommended for sensitive data that cannot be lost.
- Many businesses that deal with sensitive customer information such as payment details would most likely choose this method since that data is very critical to keep intact.

Write Back:

- Saves data to cache only.
- But at certain intervals or under a certain condition you would save data to the main memory.
- Disadvantage: there is a high probability of data loss.

5. Line Size

Another design element is the line size. When a block of data is retrieved and placed in the cache, not only the desired word but also some number of adjacent words are retrieved.

As the block size increases from very small to larger sizes, the hit ratio will at first increase because of the principle of locality, which states that data in the vicinity of a referenced word are likely to be referenced in the near future.

As the block size increases, more useful data are brought into the cache. The hit ratio will begin to decrease, however, as the block becomes even bigger and the probability of using the newly fetched information becomes less than the probability of reusing the information that has to be replaced.

Two specific effects come into play:

- Larger blocks reduce the number of blocks that fit into a cache. Because each block fetch overwrites older cache contents, a small number of blocks results in data being overwritten shortly after they are fetched.
- As a block becomes larger, each additional words is farther from the requested word and therefore less likely to be needed in the near future.

6. Number of Caches

Multilevel Caches:

- On chip cache accesses are faster than cache reachable via an external bus.
- On chip cache reduces the processor's external bus activity and therefore speeds up execution time and system performance since bus access times are eliminated.
- L1 cache always on chip (fastest level)
- L2 cache could be off the chip in static ram
- L2 cache doesn't use the system bus as the path for data transfer between the L2 cache and processor, but it uses a separate data path to reduce the burden on the system bus. (System bus takes longer to transfer data)

- In modern designed computers L2 cache may now be on the chip. Which means that an L3 cache can be added over the external bus. However, some L3 caches can be installed on the microprocessor as well.
- In all of these cases there is a performance advantage to adding a third level cache.

Unified (One cache for data and instructions) vs **Split** (two, one for data and one for instructions)

These two caches both exist at the same level, typically as two L1 caches. When the processor attempts to fetch an instruction from main memory, it first consults the instruction L1 cache, and when the processor attempts to fetch data from main memory, it first consults the data L1 cache.

7. Mapping Function

Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines

Further, a means is needed for determining which main memory block currently occupies a cache line. The choice of the mapping function dictates how the cache is organized. Three techniques can be used: direct, associative, and set-associative.

Cache vs RAM

Although Cache and RAM both are used to increase the performance of the system there exists a lot of differences in which they operate to increase the efficiency of the system.

RAM	Cache
RAM is larger in size compared to cache. Memory ranges from 1MB to 16GB	The cache is smaller in size. Memory ranges from 2KB to a few MB generally.
It stores data that is currently processed by the processor.	It holds frequently accessed data.
OS interacts with secondary memory to get data to be stored in Primary Memory or RAM	OS interacts with primary memory to get data to be stored in Cache.
It is ensured that data in RAM are loaded before access to the CPU. This eliminates RAM miss never.	CPU searches for data in Cache, if not found cache miss occur.

Differences between associative and cache memory:

Associative Memory	Cache Memory
A memory unit access by content is called associative memory.	A fast and small memory is called cache memory.
It reduces the time required to find the item stored in memory.	It reduces the average memory access time.
Here data accessed by its content.	Here, data are accessed by its address.
It is used where search time is very short.	It is used when a particular group of data is accessed repeatedly.
Its basic characteristic is its logic circuit for matching its content.	Its basic characteristic is its fast access.
It is not as expensive as cache memory.	It is expensive as compared to associative memory.
It is suitable for parallel data search mechanism.	It is useful in increasing the efficiency of data retrieval.

Advantages of Cache Memory

- The main memory is slower than cache memory.
- It creates a way for fast data transfers so it consumes less access time as compared to main memory.
- It stores frequently access that can be executed within a short period of time.

Disadvantages of Cache Memory

- It is limited capacity memory.
- It is very expensive as compared to Memory (random access memory (RAM)) and Hard Disk.

References

Reference Books:

- J.P. Hayes, “Computer Architecture and Organization”, Third Edition.
- Mano, M., “Computer System Architecture”, Third Edition, Prentice Hall.
- Stallings, W., “Computer Organization and Architecture”, Eighth Edition, Pearson Education.

Text Books:

- Carpinelli J.D,” Computer systems organization &Architecture”, Fourth Edition, Addison Wesley.
- Patterson and Hennessy, “Computer Architecture”, Fifth Edition Morgan Kaufman.

Reference Website:

- [Differences between Associative and Cache Memory – GeeksforGeeks](#)
- <https://searchstorage.techtarget.com/definition/cache-algorithm>
- <https://www.includehelp.com/cso/types-of-cache-replacement-policies.aspx>

Video Links:

- <https://youtu.be/SV7Kk1njt5c?si=ffVZ8zVOF2qW4oqk>
- https://youtu.be/wI6_dl4WjlY?si=Hoz7CndJ95pQ71Yz
- <https://youtu.be/OfqzoQ9Kw9k?si=K3S7xGMboveTzY7z>
- https://youtu.be/QZ_9Oe5E61Q?si=mslQJSaHwmd-Kbkj
- <https://youtu.be/hhLdy3J9oqg?si=CVqVMV1QaViTcp4Q>
- <https://youtu.be/VNw00047giw?si=gUGe-WSt-Hyd3kzX>
- <https://youtu.be/5LmyIpJcd9I?si=IjbnbbbzAkuldULz>